MAP07 MODÉLISATION DE FEUX DE FORÊT

April 2022

David Heurtel–Depeiges, Saad Souilmi, Pierric Spery, Germain Vivier–Ardisson, Paul Zeinaty

Remerciements

Nous souhaitons d'abord remercier le coordinateur de ce projet scientifique collectif, M. Le Corff du département de Mathématiques Appliquées de l'École polytechnique, qui nous a apporté une aide précieuse et des pistes de réflexion, notamment en réunion de cadrage ou lors la remise de la fiche d'intention et du rapport intermédiaire.

Ce PSC n'aurait pu être mené sans notre tuteur, M. Jean-Baptiste Filippi de l'Université de Corse Pasquale Paoli, qui a suivi notre travail depuis juin et qui nous a éclairé dans la définition de nos objectifs. Nous le remercions pour le soutien qu'il a apporté dans notre démarche et les échanges constructifs que nous avons eu avec.

Nous remercions enfin le commandant Biasci, chef du Bureau Opération et Instructions de l'Unité d'Instruction et d'Intervention de la Sécurité Civile n°5, travaillant au plus proche des incendies, pour nous avoir mis en contact avec M. Filippi et informé des enjeux et de l'importance de la modélisation des feux de forêts.

Table des matières

1 Motivations, état de l'art, intérêts et objectifs 4
2 Modèles, travail personnel et démarche 6
2.1 Génération d'un jeu de données simulées, travail sur le modèle physique réel 6
2.2 Modèle physique implémenté 11
2.3 Travail de Machine Learning, choix du modèle et des hyperparamètres, test de rapidité, exportation en C++ 13
2.4 Approche de l'incertitude, idées et méthodes employées 18
2.5 Approche satellite, traitement des données, CNN 21
3 Résultats, limites 29
3.1 Emulateur du modèle physique 29
3.2 Approche autour de l'incertitude 32
3.3 Approche satellite 33
4 Conclusion 35
4.1 Conduite du projet 35
4.2 Travail en équipe 36
4.3 Perspectives 36

1 Motivations, état de l'art, intérêts et objectifs

Motivations, enjeux et partenariats

Le changement climatique engendre un nombre croissant de feux de forêts, d'une intensité croissante également. La modélisation de leur évolution présente donc un intérêt double : en analyse des risques en amont des feux et en intervention pour adapter la réponse des unités chargées de les combattre.

Ce projet scientifique collectif est réalisé en partenariat avec l'Université de Corse Pasquale Paoli et, plus indirectement, avec l'Unité d'Instruction et d'Intervention de la Sécurité Civile n°5 (UIISC5, unité dans laquelle l'un des membres du groupe a effectué son stage DFHM de première année). L'UIISC5 est une unité militaire rattachée à l'Armée de Terre qui a pour vocation le renfort ou la complétion des moyens terrestres et aériens de secours, notamment en intervention sur l'île (bien qu'elle puisse être appelée à intervenir sur tout le territoire français voire à l'étranger).

L'intérêt que porte l'UIISC5 dans ce projet tient au fait que les feux de forêts constituent la majeure partie du travail de l'unité le long de l'année : un outil permettant aux sections d'intervention d'avoir une indication sur l'évolution d'un départ de feu leur permettrait d'être nettement plus efficace sur le positionnement de moyens ainsi que sur la gestion de l'incendie.

L'UIISC5 travaillant déjà en collaboration avec l'Université de Corse sur l'élaboration d'un tel outil, notre objectif est de développer une série de nouveaux modèles pour mieux quantifier la vitesse du front de flamme d'un feu de forêt en fonction des paramètres météorologiques et géographiques de la zone enflammée. Ces modèles s'ajouteront aux modèles déjà présents dans l'outil FireCaster de l'Université de Corse. Pour cela, nous allons tenter d'apporter des solutions différentes aux problèmes de modélisation existants, grâce à une approche par l'analyse de données.

État de l'art

La modélisation de la propagation des feux de forêts à l'Université de Corse est principalement axée sur le développement de modèles physiques déterministes.

Pour cela, leurs modèles s'appuient sur les paramètres influençant le plus la vitesse et la direction de propagation d'un feu : la végétation (densité, distribution, humidité), les facteurs topographiques (pente, relief, exposition des versants...), et les facteurs météorologiques (pluviométrie, température, hygrométrie, vents). Des considérations thermodynamiques permettent d'établir une équation donnant la vitesse du front de flamme en fonction de ces paramètres, et ainsi de prédire la propagation du feu.

Ces dernières années, la modélisation des incendies s'est appuyée sur plusieurs méthodes différentes. On peut distinguer principalement trois types de modèles [3] : empiriques, semi-physiques et physiques.

— Un modèle empirique est un modèle simple qui ne tient pas compte des aspects, des mécanismes physiques du phénomène étudié. Il est donc déterminé sur la base de données expérimentales ;
— Un modèle semi-empirique est basé sur le principe de la conservation de l'énergie, mais ne fait pas de distinction entre les différentes formes de transfert de chaleur ;
— Un modèle physique, lui, prend en compte les différentes formes de transfert de chaleur. Parmi ce type de modèle, l'approche dite *multiphasique* est le modèle le plus complet développé à ce jour.

Intérêts et limites de la démarche

Les intérêts de l'approche par analyse de données par rapport aux modèles précédents sont multiples :
— La modélisation physique d'un feu est très complexe et imprévisible, elle entraîne des erreurs d'approximation qui peuvent se propager ;
— Les méthodes par analyse de données peuvent être très efficaces en terme de temps de calcul, ce qui est important d'un point de vue opérationnel pour l'utilisation en intervention ;
— Les données observées sur le terrain peuvent être utilisées et permettent d'améliorer ce modèle, tandis qu'un modèle physique est figé ;
— De potentielles relations causales non comprises ou difficiles à modéliser physiquement sont prises en compte.

Cependant, une approche par analyse de données représente un compromis. En utilisant tout modèle de régression, nous devons accepter une perte en précision incompressible. Même en supposant une dépendance déterministe entre les (nombreux) paramètres physiques et la vitesse du front de flamme, il est impossible d'avoir une perte infèrieure à l'erreur d'approximation plus l'erreur d'estimation (dans ce cas, l'erreur de Bayes est nulle).

Il peut également être très difficile, donc coûteux, de trouver la solution optimale à un problème d'apprentissage statistique.

Pour finir, les modèles d'apprentissage statistique sont souvent des boîtes noires qui, si elles permettent souvent de bonnes prédictions, ne permettent pas de comprendre les raisons d'un comportement spécifique du système.

Objectifs du projet

Les objectifs initiaux du PSC étaient les suivants :

— Disposer d'un émulateur d'un modèle physique avancé de propagation de feu, qui soit plus rapide que ce modèle [2.1, 2.2, 2.3] ;
— Étudier des marqueurs de l'incertitude dans la prédiction de la vitesse du feu [2.4].

— Développer un modèle de prédiciton purement empirique de la *Rate of Spread* ou vitesse du front de flamme [2.4].
  Nous avons ajouté un nouvel objectif au cours de l'année :
— Développer un modèle de prédiction empirique à l'échelle du feu, s'appuyant uniquement sur des données réelles ou a minima avancer vers ce modèle [2.5].

2 Modèles, travail personnel et démarche

2.1 Génération d'un jeu de données simulées, travail sur le modèle physique réel

Démarche générale et motivation

La première étape de notre travail consiste à obtenir des données de propagation d'un feu de forêt pour entraîner notre futur réseau de neurones. Avant de travailler sur des données réelles, nous avons simulé ces données à partir d'un modèle physique.

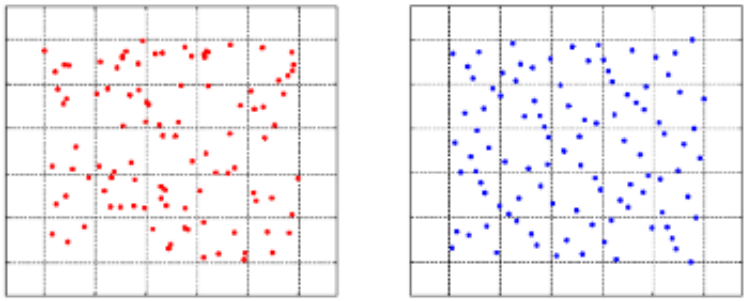
Pour obtenir nos premiers jeux de données simulées, nous avons divisé le travail en plusieurs tâches :

— construction d'une distribution de points, permettant de couvrir les plages de valeurs prises par les différents paramètres du système ;
— choix d'une méthode de stockage des jeux de paramètres générés ;
— implémentation du calcul ROS à l'aide du modèle physique de propagation fourni.

Nous étions deux membres du groupe à travailler sur cette partie du projet. En progressant en parallèle sur la première et la dernière des trois étapes décrites ci-dessus, nous avons pu nous coordonner et avons abordé la manipulation du format `csv` au même moment. Dans la dernière phase de travail clôturant cette partie du projet, nous avons travaillé de concert ce qui a accéléré la construction du format cohérent de nos fichiers `csv`, à la fois pour la lecture et l'écriture de ces derniers.

Les suites de Halton et de Hammersley

Le but étant de générer un échantillon de points "bien distribué" en haute dimension sans avoir de points trop proches, ou de large région de l'espace sans échantillon. On a opté pour des suites à discrépance faible [7], également appelées séquences *quasi aléatoires*, qui sont mieux équidistribués dans un volume donné que les nombres pseudo-aléatoires.



(a) Pseudo-random Points (b) Low-discrepancy Points

FIGURE 1 – Échantillonage d'un pavé en dimension 2 par des points pseudo-aléatoires ou à discrépance faible

La discrépance d'un ensemble de points $\{x_1, ..., x_N\}$ étant :

$$D_N(x_1, ..., x_N) = \sup_{B \in J} \left| \frac{|\{x_1, ...x_N\} \cap B|}{N} - \mu(B) \right|$$

— μ étant la mesure de Lebesgue.
— J étant l'ensemble des pavés en dimension n.

Les suites de Halton et de Hammersley [6]sont deux suites de discrépance faible basées sur la notion de l'inverse radical.

$$\forall a, b \in \mathbb{N}\ \exists! c_1(a)...c_m(a) \in \mathbb{N} \text{ telle que : } a = \sum_{i=1}^{m} c_i(a) b^{i-1}$$

Avec $c_1(a)...c_m(a)$ les uniques coefficients de a dans sa décomposition en la base b. L'inverse radicale $\Phi_b(a)$ de a dans la base b est :

$$\Phi_b(a) = \{0.c_1(a)...c_m(a)\}_{\text{base b}} \in [0, 1[$$

En base 10, $\Phi_b(a)$ devient : $\Phi_b(a) = \sum_{i=1}^{m} \frac{c_i(a)}{b^i}$. En base 2, on reconnaît la suite de Van der Corput $(\Phi_b(N))_{N \geqslant 0}$. On remarque que chaque nouvelle valeur reste loin de toutes les valeurs précédentes. Pour des N très grands, les points ont tendance à se resserrer mais en gardant une distance minimale correcte.

La discrépance de la suite de Van der Corput est en $\mathcal{O}\left(\frac{log(N)^n}{N}\right)$, ce qui est cohérent avec la majoration connue en dimension $n > 1$ qui stipule que : $D_N^*(x_1, ..., x_n) \leqslant C \frac{Log(N)^n}{N}$ avec C une constante qui dépend de la suite.

On construit la suite de Halton en dimension n de la manière suivante :

$$x_N = (\Phi_2(N), \Phi_3(N), \Phi_5(N), ..., \Phi_{p_n}(N))$$

a	Base 2	$\Phi_2(a)$
0	0	0
1	1	$0.1 = 1/2$
2	10	$0.01 = 1/4$
3	11	$0.11 = 3/4$
4	100	$0.001 = 1/8$
5	101	$0.101 = 5/8$
$\vdots$		

FIGURE 2 – Première valeurs de la suite de van der Corput.

Avec $(p_N)_{N\geqslant 0}$ la suite des nombres premiers dans l'ordre croissant. L'avantage de la suite de Halton, est que même avec ajout d'echantillons, la discrépance reste faible. La discrépance de la suite de Halton en dimension n est asymptotiquement optimale et est donc en $\mathcal{O}\left(\frac{(log(N))^n}{N}\right)$.

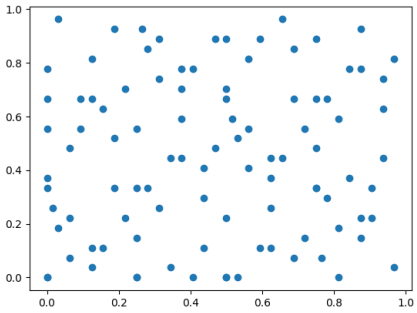


FIGURE 3 – Première simulation de la suite de Halton en dimension 2 avec 100 échantillons.

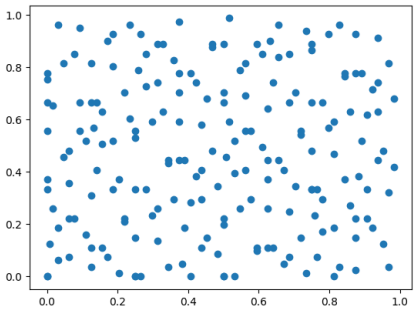


FIGURE 4 – Simulation de la suite de Halton en dimension 2 avec 200 échantillons.

La première simulation n'était pas tout à fait satisfaisante. Certains points sont trop proches, et la distribution des points n'est pas vraiment optimale. Lors de ces deux simulations on a pris

les 100 premiers et 200 premiers termes respectivement de la suite de Halton. Nous avons donc décidé d'observer le comportement de la suite si on choisissait des termes d'ordre plus haut.

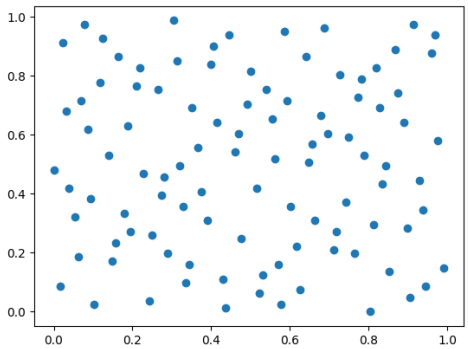


FIGURE 5 – Simulation de la suite de Halton en dimension 2 avec 100 échantillons entre x_{200} et x_{300}.

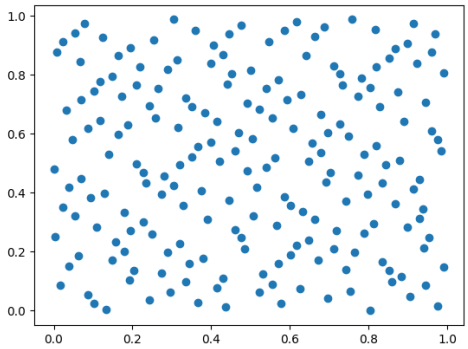


FIGURE 6 – Simulation de la suite de Halton en dimension 2 avec 200 échantillons entre x_{200} et x_{400}.

Ce résultat était bien meilleur. Cependant, nous avons décidé d'opter pour la suite d'Hammersley, assez similaire à la suite de Halton, mais qui offre des résultats plus prometteurs. Pour un nombre d'échantillons fixé N, les N termes de Hammersley en dimension n sont :

$$\forall k \in \{1, ..., N\} \;\; x_k = \left(\frac{k}{N}, \Phi_{p_1}(k), \Phi_{p_2}(k), ..., \Phi_{p_{n-1}}(k)\right)$$

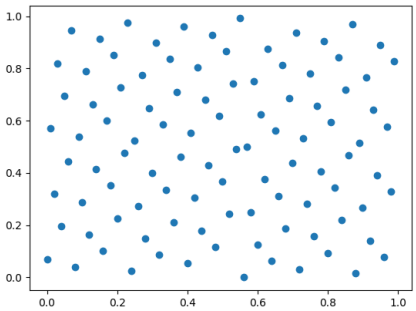


FIGURE 7 – Simulation de la suite de Hammersley en dimension 2 avec 100 échantillons entre x_{200} et x_{300}.

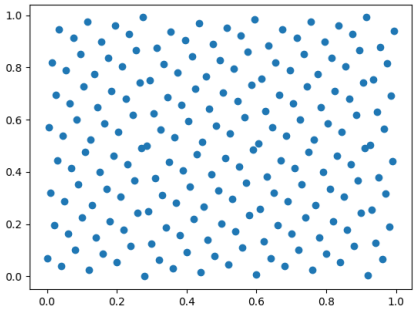


FIGURE 8 – Simulation de la suite de Hammersley en dimension 2 avec 200 échantillons entre x_{200} et x_{400}.

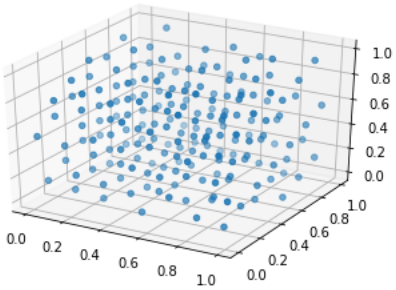


FIGURE 9 – Simulation de la suite de Hammersley en dimension 3 avec 200 échantillons entre x_{200} et x_{400}.

On observe que la distribution des points est bien meilleure et correspond mieux à nos attentes.
Afin de calculer l'inverse radicale, nous avons utilisé l'algorithme suivant :

Algorithm 1 Calculate RadicalInverse

Output : $\Phi_b(a)$
Input : $a, b \in \mathbb{N}$
$v \leftarrow 0$
$n \leftarrow 0$
while $a > 0$ **do**
  $r \leftarrow$ remainder of euclidian division of a over b
  $v \leftarrow v * b + r$
  $a \leftarrow$ quotient of euclidian divison of a over b
  $n \leftarrow n + 1$
end while
return v/b^n

2.2 Modèle physique implémenté

Après avoir obtenu un échantillonage balayant l'ensemble des paramètres, nous nous sommes intéressés au modèle physique pour générer des jeux de données de feux à partir de ces échantillons.

Présentation du modèle physique

L'objectif de cette partie est de mettre en équation la propagation du feu, c'est-à-dire de déterminer la vitesse R du front de flamme, à partir de paramètres physiques (vitesse du vent, température extérieure) et géographiques (densité du feuillage, pente du terrain).

Le modèle physique que nous avons utilisé [11] prend en compte à la fois la propagation du feu par convection et rayonnement thermique. Un bilan thermodynamique sur la zone chauffée par le feu est établi et 3 transferts thermiques sont pris en compte :

— le rayonnement dû au combustible (feuilles, branches) en fusion, ϕ_b ;
— le rayonnement dû au corps de flamme, ϕ_r ;
— la convection thermique, ϕ_c.

On considère la zone exposée au feu mais pas encore brûlée ; les trois flux de chaleur listés sèchent le combustible et peuvent réchauffer le lit de combustible jusqu'à la température d'allumage T_i. Cette étape, dite de préchauffage, est résumée dans l'équation suivante.

$$\sigma C_p R(T_i - T_a) + m\sigma C_{pw} R(T_{vap} - T_a) + m\sigma \Delta h R = a_b\phi_b + a_c\phi_c + a_r\phi_r \tag{1}$$

Elle fait intervenir la variable cible R et se réécrit plus simplement comme suit :

$$R = R_b + R_c + R_r \tag{2}$$

avec $R_b = a_b\frac{\phi_b}{\sigma q}$, $R_c = a_c\frac{\phi_c}{\sigma q}$, $R_r = a_r\frac{\phi_r}{\sigma q}$, $q = C_p(T_i - T_a) + m(C_{pw}(T_{vap} - T_a) + \Delta h)$.

Par des considérations physiques, on parvient ensuite à expliciter les flux en fonction de la vitesse du front de flamme R et des paramètres du terrain [11].

L'équation finale est la suivante :

$$\begin{aligned} R = &\min\left(\frac{S}{\pi}, 1\right)\frac{BT^4}{\beta\rho_v q} + \frac{s\Delta H}{q\tau_0}\min\left(h, \frac{2\pi}{s\beta}\right) \\ &\left(\frac{h}{2h+H}\tan\alpha + \frac{U\exp\left(-K_1\beta^{\frac{1}{2}}R\right)}{u_0}\right) + AR\frac{1+\sin\gamma-\cos\gamma}{1+\frac{R\cos\gamma}{sr_{00}}} \end{aligned} \tag{3}$$

On pourra trouver en annexe la liste de notations utilisées.

Implémentation

Dans notre démarche, nous avons commencé par travailler sur une équation simplifiée, en réduisant le nombre de dépendances en R dans les termes de l'équation. Cette simplification du problème nous a menés à implémenter deux méthodes de résolution pour comparer leurs résultats :

— la méthode du point fixe ;

— la méthode de Newton.

Les deux méthodes convergeaient vers une même valeur, indépendamment des valeurs d'initialisation de la résolution. Après quelques essais et en nous aidant du choix de M. Filipi, nous avons fixé à 200 le nombre maximal d'itérations autorisé dans chacune de ces deux méthodes.

Le nombre maximal d'itérations étant fixé (et suffisamment grand), nous avons comparé les temps d'exécution des deux méthodes pour différentes valeurs du critère d'arrêt. Les résultats sont résumés dans la figure 10.

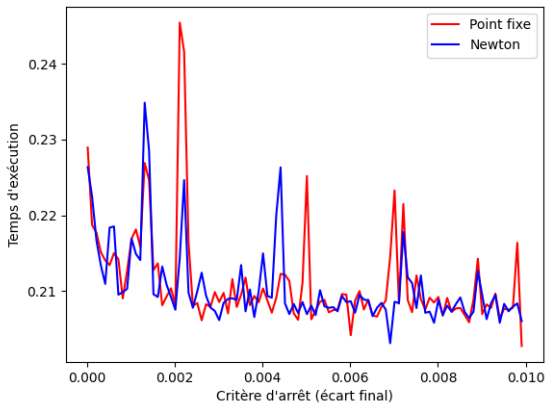


FIGURE 10 – Temps d'exécution des méthodes en fonction du critère d'arrêt

D'après le graphe, les deux méthodes sont globalement équivalentes en rapidité. En l'absence de différence majeure pour choisir l'une ou l'autre, nous avons opté pour la méthode du point fixe ; plus fiable, plus simple et déjà utilisée dans les travaux de M. Filipi.

Après ces tests préliminaires, nous avons repris l'équation exacte $R = f(R)$. Pour donner plus de visibilité au code et anticiper un large choix de données, nous distinguons 3 catégories de variables :

— les constantes (ex : capacité thermique massique, chaleur de combustion, température de changement d'état) ;
— les paramètres décrivant la nature du terrain (ex : pente, vitesse du vent, épaisseur et taux d'humidité de la couche de carburant) ;
— les autres variables, dépendant de l'évolution du feu (ex : vent, température, hauteur des flammes).

Dans notre première implémentation, les valeurs des constantes sont initialisées pour ne plus être modifiées par la suite. Elle sera ensuite modifiée pour augmenter l'efficacité du code.

Les paramètres décrivant la nature du terrain sont initialisés au début de chaque résolution à l'aide d'une fonction prévue à cet effet. En effet, à ce stade, le terrain est considéré comme homogène. Or, il est envisageable d'effectuer des simulations avec un terrain sur lequel les valeurs de ces paramètres varient en fonction de l'espace. Avec ce choix d'implémentation, nous gardons ainsi la possibilité de mettre à jour ces paramètres au cours de la résolution. Nous verrons par la suite qu'une telle implémentation s'avèrera inefficace en terme de rapidité.

Enfin, les autres variables sont directement mises à jour dans la fonction de résolution.

Cette différence d'implémentation par rapport au code proposé par M. Filippi nous a demandé un peu de travail : nous avons rencontré plusieurs erreurs lors de l'écriture du code, notamment du fait du grand nombre de variables affectant sa lisibilité. Nous avons finalement obtenu des résultats concordant en utilisant la méthode du point fixe.

Choix du stockage des jeux de données

Après avoir généré des jeux de données, nous avons choisi de les stocker ligne par ligne dans un fichier `csv`. Ce format de fichier nous a paru naturel car compact et facile à exploiter, et a facilité la manipulation des données au sein du groupe et la mise en oeuvre des tests.

2.3 Travail de Machine Learning, choix du modèle et des hyperparamètres, test de rapidité, exportation en C++

Nous avons tout d'abord testé sur des jeux de données simulées de plusieurs tailles deux approches afin de vérifier la faisabilité de nos objectifs. Nous avons alors rapidement utilisé deux familles de modèles : Réseaux de neurones *classiques* et Machines à vecteur de support. Les deux familles de modèles donnaient des résultats satisfaisants, mais plus précis pour les

réseaux de neurones choisis. Nous détaillons dans cette partie le travail plus spécifique sur ces modèles.

Nous disposons de 9 jeux de données simulées avec de 2^7 à 2^{15} occurrences. Chacun de ces jeux de données contient 10 colonnes. Les 9 premières correspondent aux 9 variables physiques prises en compte par le modèle *Balbi2020*. Elles forment les variables explicatives. La 10ème colonne contient la variable cible, à savoir la vitesse du front de flamme ou *RoS*. Le choix des tailles n'est pas anodin. Pour les plus petites valeurs, il s'agit de la taille approximative d'un jeu de donnée obtenu en laboratoire. Pour les plus grandes valeurs, il s'agit des jeux de données les plus gros permettant un calcul sur nos machines ou celles des salles informatiques sans que des messages d'erreurs interrompent le processus.

Nous avons testé sur ces jeux de données trois types de modèles de machine learning : en premier lieu des réseaux de neurones, puis des machines à vecteur de support et enfin des forêts aléatoires.

Choix automatique des hyperparamètres

Le choix des hyperparamètres d'un modèle d'apprentissage statistique est primordial. Il permet d'obtenir une précision plus élevée sur le résultat final. L'enjeu pour nous était double. Il s'agissait d'abord d'identifier un modèle précis et simple pour disposer d'un émulateur du modèle physique réel. Ensuite, l'étude rapide du meilleur modèle en fonction de la taille du jeu de données permettra de mieux sélectionner le modèle pour une approche totalement empirique.

Chaque famille de modèle prend des hyperparamètres différents. Les plus connus sont, pour des réseaux de neurones : la géométrie du réseau (nombre et taille des couches cachées), la fonction d'activation, une éventuelle pénalisation des coefficients pour éviter l'*overfitting*. Pour une machine à vecteur de support, le type de *kernel* choisi ainsi que des paramètres pour ce *kernel* viennent tout de suite à l'esprit ; mais il faut aussi rajouter une pénalisation de régularisation et la marge d'erreur de la machine à vecteur de support [12]. Les paramètres en question varient peu d'une implémentation à l'autre des familles de modèles et nous nous référerons à la documentation des bibliothèques choisies.

Un autre type d'hyperparamètres concerne l'algorithme d'optimisation qui calcule le modèle à partir des données d'apprentissage. Le choix de cet algorithme (en général, pour des réseaux de neurones, une variation de la descente de gradient stochastique) influence la solution trouvée ainsi que sa performance. Ici encore, nous nous référerons principalement à la documentation des bibliothèques choisies ainsi qu'à différentes résultats de recherche pour limiter l'étendue de l'optimisation que nous mènerons (aux hyperparamètres les plus importants) [10].

Pour finir, il existe plusieurs méthodes permettant d'explorer l'espace des hyperparamètres : GridSearch, Random Search, méthodes bayésiennes...Nous avons utilisé les trois sur plusieurs familles de modèles et nous avons implémenté à la main GridSearch et RandomSearch.

GridSearch correspond à une méthode de type "brute force" pour déterminer le choix optimal d'hyperparamètres. A partir d'une liste pour chacun des hyperparmètres des valeurs que nous voulons tester, nous générons le produit cartésien de ces listes (cela forme une grille dans l'espace des hyperparamètres) et nous calculons pour chaque choix d'hyperparamètres le score du modèle. Cela correspond à calculer la valeur optimale d'une fonction (le score) en discrétisant l'espace de départ.

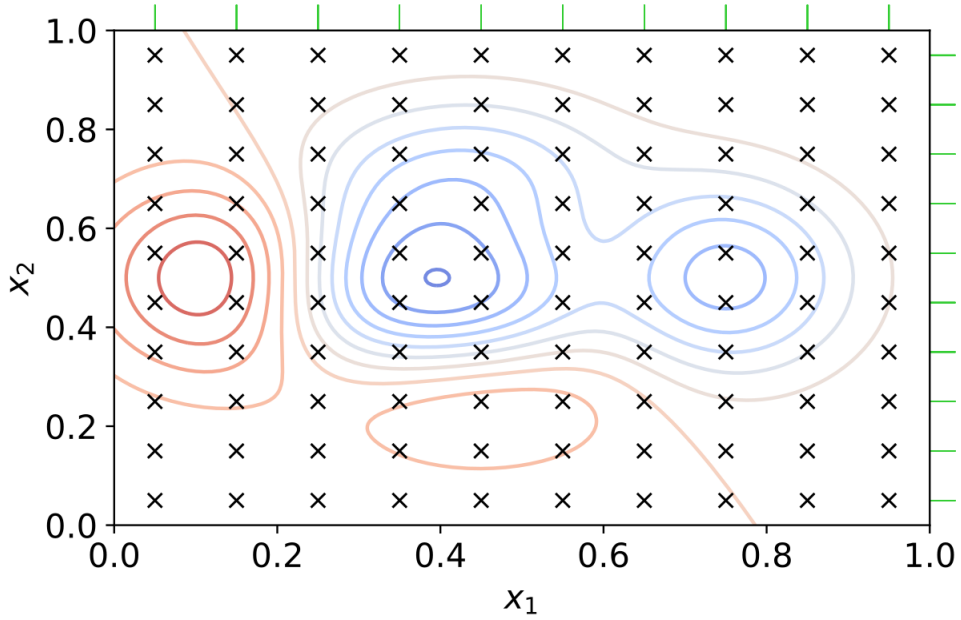


FIGURE 11 – Exemple de GridSearch en 2D, x_1 et x_2 étant les hyperparamètres à optimiser / By Alexander Elvers - Own work, commons.wikimedia.org

Ici, par exemple, si l'on souhaite minimiser la perte (le rouge représentant de grandes valeurs et le bleu des petites), la solution optimale est environ $(0.4, 0.5)$ et l'algorithme retournera l'un des quatre points les plus proches.

Une alternative (et qui se rapproche des méthodes utilisées pour explorer l'espace des paramètres physiques) est RandomSearch. Il s'agit là aussi d'une méthode "brute force" mais au lieu de spécifier une liste de valeurs, l'algorithme prend en entrée une mesure de probabilité sur chacun des hyperparamètres (ou une mesure de probabilité sur l'espace produit) et tire au hasard un certain nombre de points sur lesquels il va calculer le score. Un telle méthode peut être utile si l'on souhaite explorer l'espace des hyperparamètres avec moins de points.

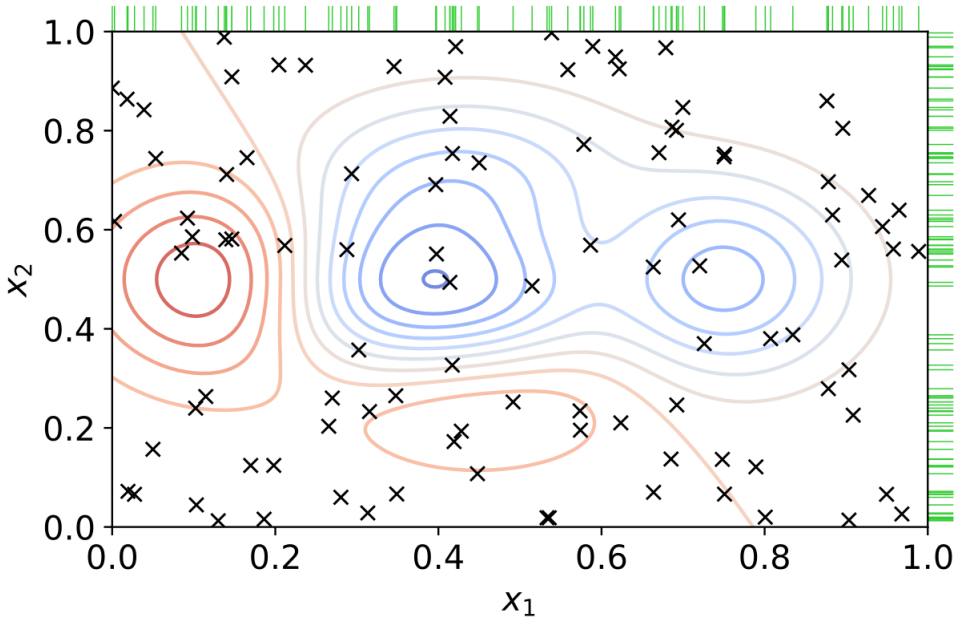


FIGURE 12 – Exemple de RandomSearch en 2D / By Alexander Elvers - Own work, commons.wikimedia.org

Pour finir, il existe plusieurs types de méthodes dites "Bayésiennes" ou "adaptatives". Celles-ci calculent le score sur plusieurs points avant de choisir un nouvel ensemble de points. Elles procèdent de manière itérative pour arriver à une solution optimale. A noter que des méthodes "descente de gradient" sont possibles avec une petite adaptation à faire pour des hyperparamètres de type catégorie (fonction d'activation, kernel, solver) ou entier (taille du réseau de neurones) et pas de type réel.

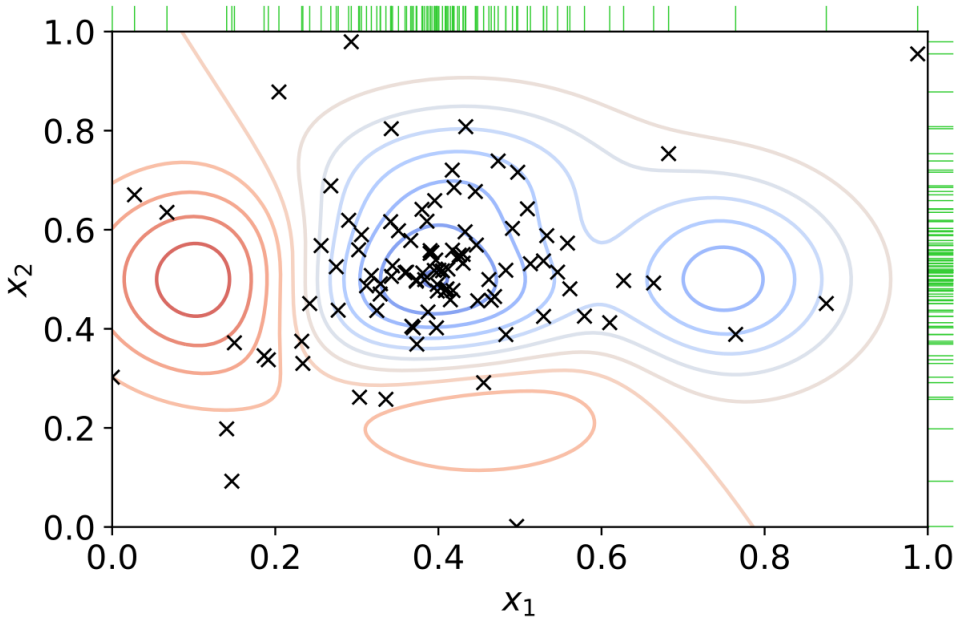


FIGURE 13 – Exemple de méthode bayésienne en 2D / By Alexander Elvers - Own work, commons.wikimedia.org

Nous avons utilisé GridSearch pour les réseaux de neurones que nous avons choisis (*Multi-Layer Perceptron Regressor* de *Scikit-learn*). Pour les machines à vecteurs de support, nous avons appliqué GridSearch et des méthodes bayésiennes. RandomSearch a pour sa part été utilisés sur les forêts aléatoires pour explorer la faisabilité de cette méthode. La demande d'explorer des méthodes autour des forêts aléatoires venait de notre coordinateur. Celui-ci nous les conseillait puisque des modèles de bruits (et donc d'incertitude) étaient déjà connus.

Test de rapidité et premiers résultats

Comme suggéré par notre tuteur de PSC et conformément à l'un de nos objectifs, nous avons cherché en premier lieu un émulateur du modèle physique Balbi2020. Pour que cet émulateur soit efficace, il faut non seulement qu'il soit précis mais également qu'il soit plus rapide que le modèle physique d'origine [13].

Dès que nous avons disposé d'un modèle avec un score convenable, nous avons testé sa rapidité contre celle du modèle physique.

Les tests ont été menés à l'aide de la fonction `repeat` du module `timeit`. Voici les opérations comparées, avec en entrée un fichier `csv` entier de jeux de données simulées de taille arbitraire :

— **pour le modèle physique**, le temps de résolution de l'équation physique modélisant la propagation (cf 2.2)

— **pour le réseau de neurones**, le temps de prédiction du ROS (le prédicteur étant donné).

Les premiers tests effectués ont donné les résultats suivants :
Temps d'exécution minimal obtenu sur 2×1000 *itérations de calcul du ROS sur le jeu de données simulées* `Jeu_donnees_out_128.csv`
`test_Physique: 0.01536180000000087`
`test_ResNeur: 0.0004530999999872165`

La différence des temps d'exécution nous a semblé excessive ; nous supposions qu'elle venait d'une surestimation du temps de calcul du modèle physique. Pour remédier à cela, nous avons grandement simplifié l'implémentation de la méthode du point fixe, au détriment de la modularité du code recherchée initialement (liberté de faire varier les paramètres de paysage).

Après ces modifications, le modèle physique est devenu plus rapide comme le montrent les résultats :
Temps d'exécution minimal obtenu sur 2×1000 *itérations de calcul du ROS sur le jeu de données simulées* `Jeu_donnees_out_128.csv`
`test_Physique: 0.0010552999999999813`
`test_ResNeur: 0.0004084999999918182`

Dans les faits, les deux opérations comparées ne sont pas de la même nature (explications plus poussées en 3.1). En particulier, nous soupçonnons que le langage interprété que nous avons utilisé ne biaise les temps d'exécution. C'est pourquoi nous avons souhaité retranscrire les deux programmes utilisés (codés jusque là en Python) en C++, pour reproduire les tests et vérifier leur cohérence.

Lors de l'écriture de ce rapport, nous étions toujours en train de retranscrire les programmes en question en C++ pour comparer leur rapidité.

2.4 Approche de l'incertitude, idées et méthodes employées

Jeu de données

Ne disposant pas d'un jeu de données réelles avec du bruit, nous avons décidé de générer notre propre jeu de donnée perturbé. La méthode avec laquelle nous avons généré ce jeu de donnée permettra en plus d'évaluer la sensibilité de la *RoS* en fonction des différents paramètres.

En effet, notre algorithme de génération du jeu de données perturbé fonctionne ainsi. Il récupère un point dans l'espace des paramètres physique, le perturbe aléatoirement suivant chacune de ses composantes suivant une certaine loi (dans notre cas, loi normale avec écart type proportionnel à la valeur en question, autrement dit, incertitude relative constante). Ensuite on calcule la valeur de la vitesse en ce point là avec le modèle physique. On dispose ainsi d'un jeu de donnée permettant de modéliser l'incertitude lié à la mesure physique des paramètres de départ. Par exemple, on peut visualiser ci-dessous l'impact d'une incertitude de 10% sur la vitesse du front de flamme pour un point au milieu de notre espace des paramètres physiques.

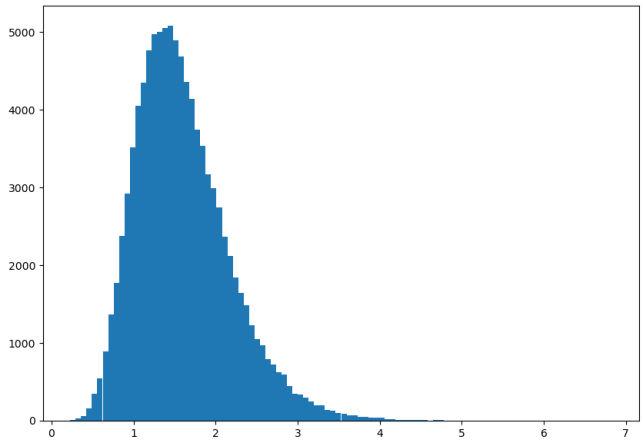


FIGURE 14 – Impact d'une incertitude de 10% sur chacune des coordonnées sur la vitesse du front de flamme pour un point avec les caractéristiques suivantes : [Pente=0.5 rad,Température de l'air=$300K$, Capacité thermique spécifique du combustible=$100J.K^{-1}.Kg^{-1}$, épaisseur du combustible=$20cm$, humidité de 50%, Vent=$30m.s^{-1}$]

La figure ci-dessus montre la distribution approchée (histogramme) calculée manuellement pour une incertitude relative (écart type) de 10% pour un point au milieu de notre espace des paramètres. Concrètement, on visualise ici la distribution de probabilité (à un facteur près) de $RoS(X)$ où X suit une loi normale de centre le point P en question et de matrice de covariance diagonale avec pour variance $(0.1 \times P_i)^2$. Le choix d'une incertitude relative constante est purement arbitraire et à des fins d'illustration. La distribution de probabilité en question est intéressante en elle même et il est possible de visualiser la sensibilité du modèle à une incertitude suivant un ou plusieurs paramètres dans une situation donnée. Cette visualisation peut en elle même fournir une première estimation des facteurs de risque pour une situation réelle.

L'objectif est de pouvoir faire varier cette incertitude pour permettre à un outil d'apprentissage statistique d'apprendre la distribution ou a minima des quantiles. In fine, avec un jeu de donnée réelles fortement bruité, les méthodes développées permettraient idéalement d'obtenir encore des quantiles corrects (après réentraînement bien sûr).

Nous disposons donc maintenant d'un ensemble de jeu de données bruitées de différentes tailles (comme pour le jeu de données simulées d'origine).

Régression quantile

Par souci de simplicité et pour réutiliser des bibliothèques déjà existantes, nous avons d'abord effectué une régression quantile sur le jeu de données perturbé. Une régression quantile tente de déterminer, parmi une famille de fonction donnée laquelle est optimale pour décrire le α-ième quantile. La fonction de coût est la fonction *pinball*, décrite plus bas.

Classification SoftMax

Nous avons tenté ici une autre méthode, plus artisanale pour obtenir la distribution de la vitesse en fonction des paramètres physiques. L'idée est, plutôt que de demander quelle est la valeur de la vitesse, de demander si la vitesse appartient à un certain intervalle et de le faire pour tout une série d'intervalle couvrant l'ensemble des valeurs possibles de la vitesse. Concrètement, on modifie le jeu de donnée en remplaçant la vitesse par l'intervalle auquel appartient la vitesse. Puis on entraîne un classificateur de type soft-max. La particularité d'un classificateur de type soft-max est qu'il retourne un vecteur de la taille du nombre de classe avec dans chaque case un nombre réel compris entre 0 et 1. Avec une fonction de perte appropriée, ce nombre représente la probabilité que la variable cible (ici la vitesse) appartienne à la classe en question (ici un intervalle pour cette vitesse).

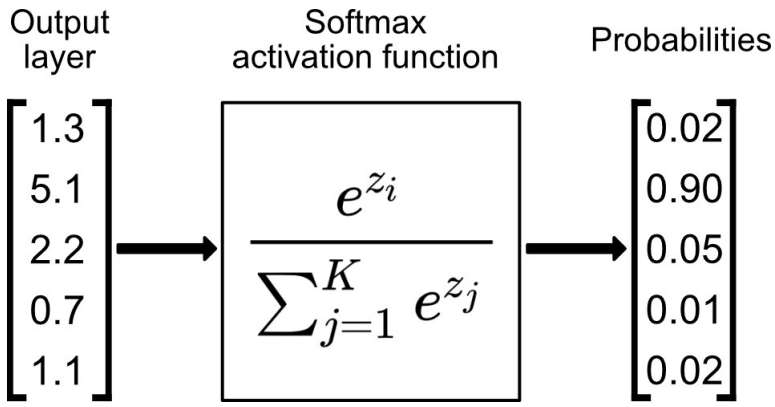


FIGURE 15 – Principe du classificateur soft-max avec application de cette fonction à la dernière couche. Dario Radecic, towardsdatascience blog

Autres méthodes

Nous avions évoqué dans notre rapport intermédiaires plusieurs méthodes, certaines relevant de l'état de l'art, d'autres moins utilisées ou plus récentes (*NGBoost* par exemple). Nous n'avons pas eu le temps de les explorer davantage. En effet nous avons priorisé l'obtention d'un émulateur, puis de données réelles et enfin une première étude des facteurs d'incertitude.

2.5 Approche satellite, traitement des données, CNN

Démarche générale et motivation

Nous avons ensuite décidé de travailler sur des données satellites réelles. Notre motivation initiale était d'avoir plus de données pour notre émulateur, et le fait que ces données soient réelles permettait de le rendre plus performant.

Après avoir traité ces données pour les utiliser, nous avons eu l'idée de tenter une nouvelle approche pour prédire la propagation de feux de forêts, en nous basant sur ces données satellites réelles.

Notre idée était la suivante :

— obtenir des données satellites d'évolution de feux de forêts ;
— obtenir des données géographiques et climatiques dans la zone et au moment de ces feux ;
— entraîner un algorithme de machine learning, prenant en entrée un feu réel à l'instant t et les données géographiques et climatiques de ce feu, et en sortie ce feu à l'instant t+1.

Cet algorithme doit pouvoir déterminer l'évolution future d'un feu à l'instant t+1, connaissant toutes les données à l'instant t.

L'avantage de cette approche est qu'elle pourrait nécessiter moins de temps de calcul pour la prédiction des feux que le modèle physique. De plus, si l'algorithme fournit de bons résultats, cette méthode nouvelle pourrait être très pratique et utilisable par différentes unités d'incendies. Elle présenterait également l'avantage de pouvoir prendre en compte des phénomènes différents de la propagation "standard" comme celle étudiée par les modèles physiques ou les expériences en laboratoire : par exemple, la propagation des feux de forêts californiens crée un système météorologique propre qui s'auto-entretient, accélère la propagation des feux à des températures beaucoup plus élevées que celles des feux traditionnels. A ceci s'ajoutent beaucoup plus d'aspects chaotiques de la propagation.

C'est pourquoi nous pensons qu'une approche géographique à plus grande échelle pourrait fonctionner, avec des outils comme les réseaux de neurones convolutifs. En effet, il est logique que la propagation du feu soit en premier lieu influencée par les paramètres physiques locaux et ensuite par des paramètres d'échelle à la taille du feu. Des réseaux de neurones convolutifs ont déjà été appliquées à la prédiction des précipitation sur un schéma relativement similaire à ce que nous envisagions ici. Il s'agissait, pour être plus précis, de réseaux de neurones de type ConvLSTM [8].

Nous avons eu cette idée en février, en parallèle de nos autres travaux. Ce travail n'étant pas prévu initialement, l'obtention de données météorologiques et géographiques à un instant donné s'est avéré compliquée et très riche en stockage. Une autre difficulté est la non régularité des prises de vue satellite. Nous ne disposons pas de photographie satellite du feu toutes les heures par exemple.

Présentation des données

Les données satellites sont issues de bases de données de la NASA [15] et contiennent les coordonnées de points géographiques où des feux ont été détectés, à une date et une heure données, en Californie. Nous utiliserons les colonnes *latitude*, *longitude*, *acq-date* et *acq-time*.

Table 1: Exemples de points d'un fichier de données satellites

latitude	longitude	brightness	acq-date	acq-time
29.980009	-81.383148	299.2	2021-07-01	630
30.940664	-88.01963	297.17	2021-07-01	630
31.145298	-87.990707	301.2	2021-07-01	630
31.152466	-87.990257	316	2021-07-01	630

Les dates sont en format YYYY-MM-DD et les heures en HHMM.

Algorithmes de clustering

Les données ne sont pas groupées par feux. Un satellite va détecter à un instant donné tous les points chauds dans une zone définie. Il faut regrouper les points proches géographiquement, correspondant à un même feu.

Pour cela, nous allons utiliser un algorithme de clustering. Il en existe plusieurs types. Ici nous ne connaissons pas le nombre de feux, donc les algorithmes prenant en entrée un nombre défini de clusters souhaité (ex : algorithme des K-Moyennes) ne sont pas utilisables. Les feux peuvent aussi avoir des formes variées, et il y a beaucoup de points qui ne sont que du bruit. Nous avons donc décidé d'utiliser un algorithme à densité, le « density-based spatial clustering of applications with noise » ou $DBSCAN$ [1]. Cet algorithme a la particularité de repérer les points correspondant à du bruit. Ci-dessous est présenté le fonctionnement de l'algorithme et les fonctions auxquelles il fait appel : $epsilonVoisinage$ et $etendreCluster$.

Cet algorithme est implémenté dans la bibliothèque *sklearn.cluster*. L'algorithme DBSCAN prend en entrée 2 paramètres : la distance ϵ et le nombre minimum de points $MinPts$ devant se trouver dans un rayon ϵ pour que ces points soient considérés comme un cluster. L'idée de l'algorithme est la suivante : pour un point donné, récupérer ses voisins à une distance ϵ et vérifier qu'il y a $MinPts$ points ou plus. Ce point est alors considéré comme faisant partie d'un cluster. On parcourt ensuite l'ϵ-voisinage de proche en proche afin de trouver l'ensemble des points du cluster.

L'algorithme DBSCAN renvoie une liste d'entiers contenant autant d'éléments que de points satellites. -1 correspond à un point bruit, et les clusters sont ensuite indexés à partir de 0.

Algorithm 2 epsilonVoisinage

Output : PtsVoisins, tous les points de D qui sont à une distance inférieure à ϵ de P
Input : Donnees D, Point P, ϵ

Algorithm 3 etendreCluster

```
Input : D, P, PtsVoisins, C, ϵ, MinPts
ajouter P au cluster C
loop
  pour chaque point P' de PtsVoisins
  if P' n'a pas été visité then
    marquer P' comme visité
    PtsVoisins' ← epsilonVoisinage(D, P', ϵ)
    if tailleDe(PtsVoisins') >= MinPts then
      PtsVoisins = PtsVoisins U PtsVoisins'
    end if
  end if
  if P' n'est membre d'aucun cluster then
    ajouter P' au cluster C
  end if
end loop
```

Algorithm 4 DBSCAN

```
Output : une répartition des points en clusters et en bruit
Input : D, ϵ, MinPts
C ← 0
loop
  Pour chaque point P non visité des données D
  marquer P comme visité
  PtsVoisins ← epsilonVoisinage(D, P, ϵ)
  if tailleDe(PtsVoisins) < MinPts then
    marquer P comme BRUIT
  else
    C ← C + 1
    etendreCluster(D, P, PtsVoisins, C, ϵ, MinPts)
  end if
end loop
```

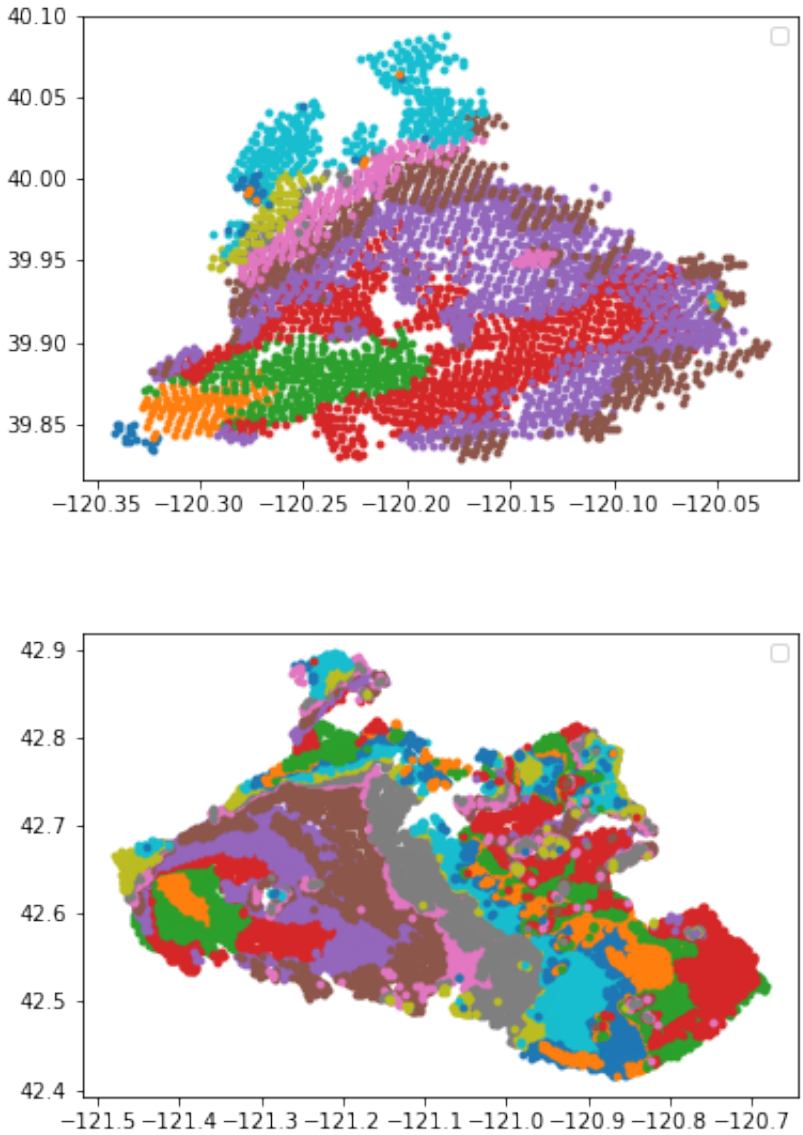


FIGURE 16 – Exemples de feux obtenus grâce à l'algorithme DBSCAN

On peut ainsi afficher un feu clusterisé grâce à l'algorithme [Fig. 16]. Pour simplifier la compréhension, chaque couleur correspond à une prise satellite d'un même horaire. On peut ainsi voir la propagation d'un feu. Par souci de clarté, nous n'avons pas affiché la légende indiquant pour chaque couleur la date et l'heure de la prise satellite. L'abscisse correspond à la longitude et l'ordonnée à la latitude. On voit sur ces deux exemples que les feux peuvent avoir des tailles très différentes.

Cet algorithme présente quelques inconvénients : les paramètres à ajuster dépendent de la densité des feux, qui n'est pas fixée. Si ϵ est trop petit ou $MinPts$ trop grand, les clusters ne

pourront pas se propager par densité, il y aura beaucoup de petits clusters pour un seul feu et beaucoup de points seront considérés comme du bruit. À l'inverse, si ϵ est trop grand ou $MinPts$ trop petit, l'algorithme se propagera trop simplement et les feux ne seront pas assez denses.

Utilisation de l'algorithme et optimisation des paramètres

On applique l'algorithme aux coordonnées géographiques des feux. Pour optimiser les paramètres à notre problème, on choisit une zone dans laquelle il est connu qu'il y a un feu et on créé une liste avec tous les points satellites détectés dans cette zone.
On balaye ensuite plusieurs valeurs de ϵ et $MinPts$. On cherche des paramètres tels que :
- ϵ soit assez petit et $MinPts$ assez grand pour avoir des feux denses ;
- il n'y ait qu'un un seul cluster pour ce feu ;
- il n'y ait pas de bruit.

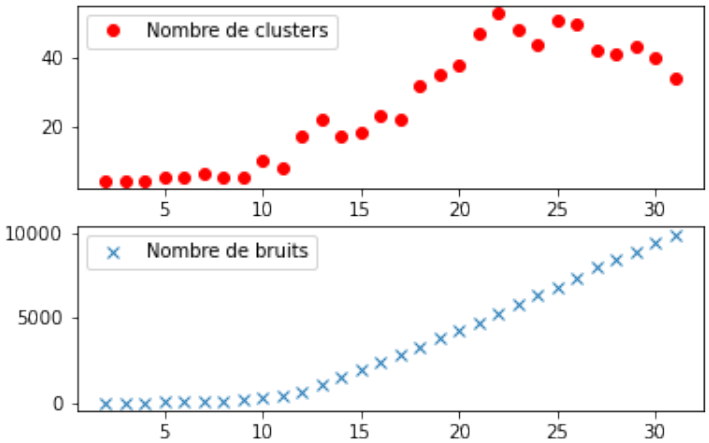


FIGURE 17 – Nombre de clusters et de bruits en fonction de $MinPts$ pour $\epsilon = 0.007$

On remarque que si ϵ est trop petit ($\epsilon = 0.007$ [Fig. 17]), quand $MinPts$ augmente il y a rapidement plusieurs clusters qui sont détectés et beaucoup de point sont considérés comme du bruit.

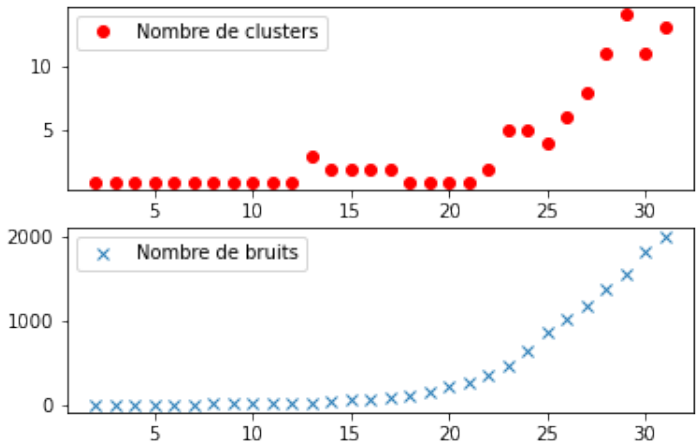


FIGURE 18 – Nombre de clusters et de bruits en fonction de $MinPts$ pour $\epsilon = 0.010$

Pour $\epsilon = 0.010$ [Fig. 18], on a bien un seul cluster et pas de points considérés comme du bruit pour $MinPts = 10$ par exemple. Ce choix permet de garder les feux denses tout en s'assurant que le modèle ne détecte pas de bruit.

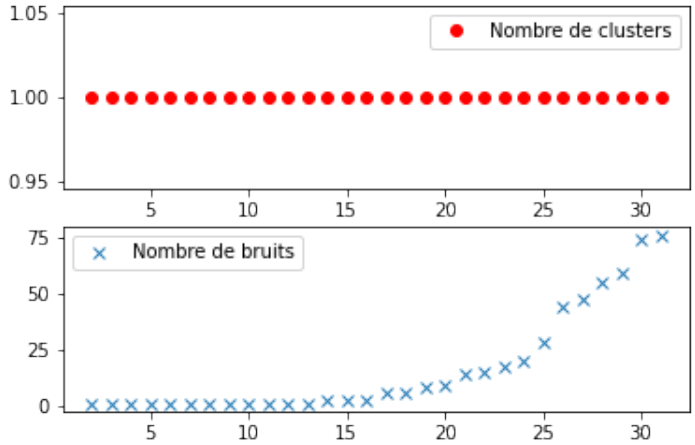


FIGURE 19 – Nombre de clusters et de bruits en fonction de $MinPts$ pour $\epsilon = 0.013$

Si ϵ est trop grand ($\epsilon > 0.013$ [Fig. 19]), on a bien un seul cluster, mais l'algorithme risque de détecter des feux pas assez denses en points et que l'on ne pourra pas exploiter.

Nous avons testé les paramètres $\epsilon = 0.01$ et $MinPts = 10$ sur l'ensemble des données et l'algorithme a renvoyé des clusters assez denses pour être exploités.

Discrétisation des données

Les points satellites sont des coordonnées (latitude et longitude). Pour la suite, il va être nécessaire de quadriller la carte en pixels pour exploiter les feux.

On créé une fonction $quadrillage(feu, origine, longueur, pas)$ qui quadrille la zone où se trouve le feu, en un carré ayant pour sommet inférieur gauche le point de coordonnées *origine*, pour côté *longueur* et *pas* est la longueur du côté d'un pixel.

On fait aussi tourner un méthode simple de "lissage" pour éviter les imprécisions du satellite. L'algorithme considère qu'un pixel est enflammé si ses voisins Est et Ouest ou Nord et Sud sont enflammés [Fig. 20]. On le fait tourner deux fois pour prendre en compte les nouveaux pixels enflammés.

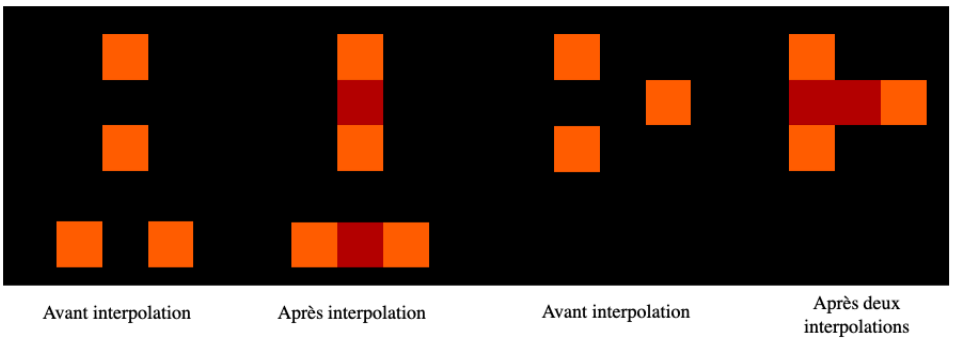


FIGURE 20 – Méthode de lissage des données satellites, appliquée deux fois lors du quadrillage

Après plusieurs essais sur des feux, cela permet d'avoir des feux plus lisses et de réduire les imprécisions dues aux mesures ponctuelles des satellites.

Le code couleur est le suivant :
- Un pixel blanc correspond à un point qui est en train de brûler à l'instant t ;
- Un pixel jaune correspond à un point qui est considéré comme en train de brûler à l'instant t par interpolation ;
- Un pixel orange correspond à un point qui a précédemment brûlé ;
- Un pixel rouge correspond à un point qui a été interpolé précédemment ;
- Un pixel noir correspond à un point qui n'a jamais brûlé.

On peut voir en [Fig. 21] que les pixels rouges obtenu par lissage ont assez bien "rempli" le feu et compensé les imprécisions du satellite.

Figure 21 – Evolution d'un feu de forêt - 1

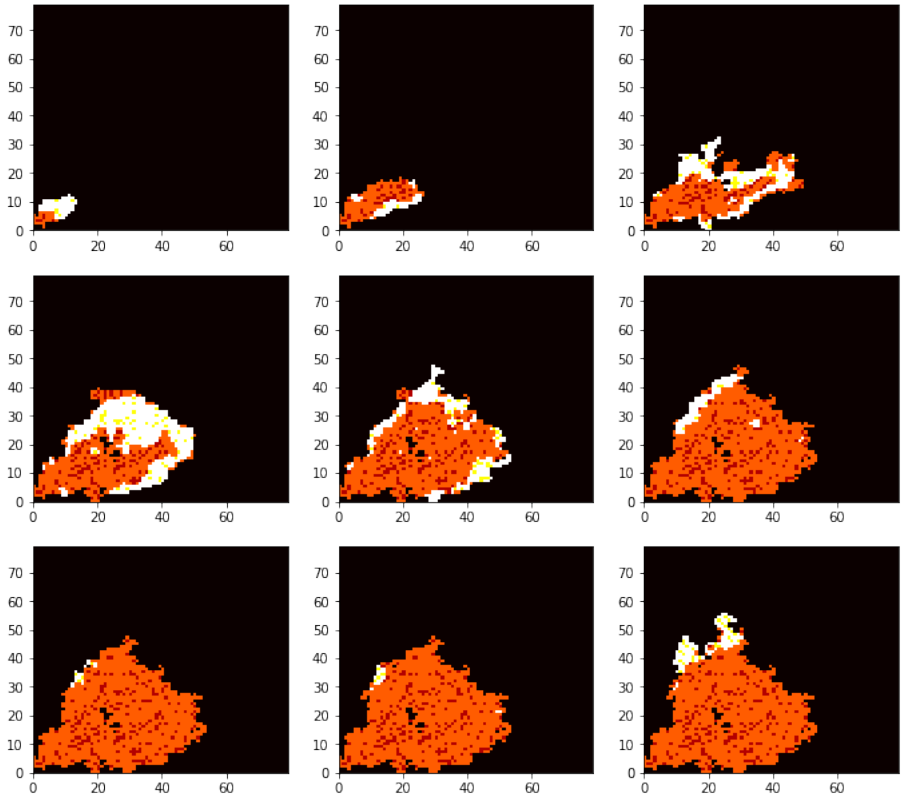
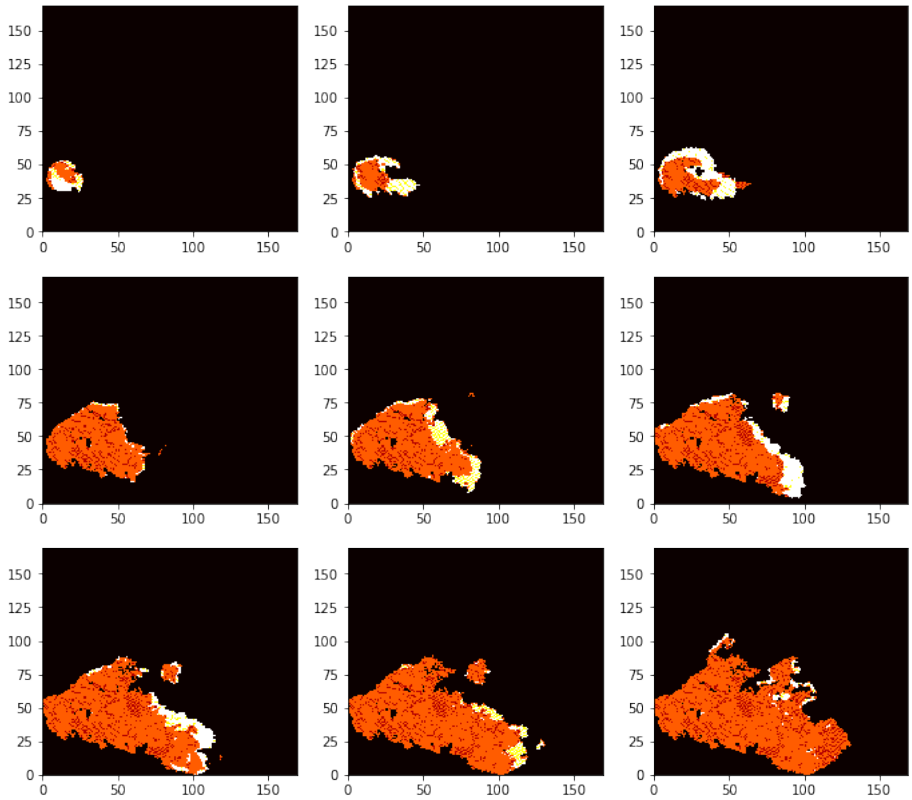


Figure 22 – Evolution d'un feu de forêt - 2



En abscisse et ordonnée sont indiqués les numéros de pixels, l'origine étant en $(0, 0)$. Ici nous avons considéré des pixels carrés de côté 500 mètres. En appliquant la fonction de quadrillage aux feux présentés en exemple en [Fig. 16], on obtient l'évolution de ces feux de forêts. Chaque image correspond à une prise satellite. Ci-dessus ont été sélectionnés 9 images parmi les 50 dont on disposait pour ces feux.

L'apparition à l'image 5 en [Fig. 24] d'un nouveau départ de feu éloigné du feu principal peut s'expliquer par une braise portée par le vent qui produit un nouveau départ de feu, en aval du vent. Ce genre de phénomène est un des facteurs prédominant dans les pertes matérielles pour les feux de forêts californiens et australiens. Cela représente également un facteur de risque supplémentaire pour les services d'urgence luttant sur le terrain.

La bibliothèque Meteostat

Pour la collecte des données météorologiques, nous avons utilisé la bibliothèque python open source Meteostat [14]. Cette bibliothèque récolte des observations et des statistiques obtenues par différentes plateformes publiques, pour les renvoyer sous format Pandas. Il est alors possible de faire des requêtes de données météorologiques dans des zones précises et à des dates choisies.

Cette bibliothèque nous donnait accès à des informations météo au niveau de certaines stations. Il nous a fallu choisir avec attention plusieurs paramètres pour permettre une interpolation efficace des données entre les stations météo. En particulier, il n'y avait pas beaucoup de station météo dans la partie de la Californie où se propagent les feux.

3 Résultats, limites

3.1 Emulateur du modèle physique

Choix des hyperparamètres

Pour les réseaux de neurones, nous avons mené la série de tests suivante :

— **Fonction d'activation** Nous avons d'abord testé sur plusieurs jeux de données et plusieurs géométries de réseau les différentes fonctions d'activation qui étaient proposées. Celles-ci étaient : ReLU (***Re**ctified **L**inear **U**nit*), tangente hyperbolique et sigmoïde (les deux dernières sont équivalentes à une fonction affine près). La conclusion de ces tests est que les trois fonctions sont équivalentes en terme de score. Cependant, les réseaux avec ReLU n'ont présenté aucun problème de convergence là où ceux avec les autres fonctions d'activation oui. Cela illustre, peut-être le problème du *vanishing gradient* [2]. Nous choisirons pour la suite ReLU comme fonction d'activation. Ce choix semble standard à ce jour.

— **Solver, algorithme d'optimisation et *learning rate***, le choix de l'algorithme d'optimisation est lui aussi important. Comme montré dans la table 5 en annexe, sur des

réseaux de neurones à une couche cachée, L-BFGS est bien plus performant que SGD (*stochastic gradient descent*) ou Adam sur des jeux de données de taille importante. En revanche, L-BFGS est beaucoup plus lent (SGD est particulièrement rapide). Nous verrons par la suite que le nombre maximum d'itérations, s'il est important pour qu'il n'y ait aucune alerte de convergence, n'offre que des améliorations marginales (pour L-BFGS) pour de grandes valeurs alors qu'il augmente linéairement le temps de calcul.

— **Taille optimale du réseau**, en fonction de la taille du jeu de données. L'enjeu de l'approche purement empirique est différent selon la taille du jeu de données. Pour un petit jeu de données, il est important que le modèle puisse généraliser avec précision. Pour un gros jeu de données, il est important que le modèle puisse mémoriser avec précision. Comme le montrent les tableaux en annexe, pour un petit jeu de données, un réseaux de neurone peu large mais plus profond (3 couches cachées à 20 neurones chacune) est plus efficace qu'un réseau de neurone peu profond mais large (1 couche cachée à 100 voire 200 neurones). Pour un jeu de données plus important, un réseau peu profond mais large semble beaucoup plus performant (nous n'avons pas pu tester autant de géométrie de réseaux pour les plus gros jeux de données étant donnée la quantité de calcul nécessaire). Notre travail s'est donc porté davantage sur les petits jeux de données pour déterminer quel type de modèle serait le plus adapté avec un jeu de données réels, de petite taille donc. Qui plus est, la principale utilité d'étudier les jeux de grande taille était de pouvoir disposer d'un émulateur du modèle physique, ce qui est chose faite étant donné la précision obtenue sur certains réseaux de neurones avec le plus gros jeu.
— **Impact du paramètre de régularisation** Le paramètre de régularisation est censé forcer le réseau à apprendre à généraliser [9]. Il est donc logique que sur les réseaux de neurones entraînés à mémoriser (donc ceux des plus gros jeux), l'impact de ce paramètre soit négatif et qu'une augmentation du paramètre engendre une baisse du score. D'une manière plus surprenante, c'est aussi le cas pour les réseaux de neurones les plus profonds qui sont censé mieux généraliser. Un autre aspect qui n'est pas visible dans les simples résultat de score est le temps de calcul. Un paramètre de régularisation plus élevé impliquait souvent un temps de calcul plus important.

— **Impact du maximum d'itération** Sans surprise, le nombre maximum d'itération de l'algorithme d'optimisation améliore la précision du résultat final. Ceci étant dit, l'amélioration est parfois marginale pour un temps de calcul bien plus élevé (c'est la cas souvent avec le *solver* L-BFGS.

Vous trouverez en annexe une partie des résultats d'optimisation des hyperparamètres sur lesquels se base cette partie. Nous vous ferons également parvenir le reste des chiffres en question avec le code ayant permis ces calculs.

Pour les machines à vecteur de support, nous avons testé, par méthode bayésienne :
— **le kernel**
— **la marge** ϵ
— **le paramètre de régularisation**

Ce test a permis une nette amélioration par rapport à la machine à support de vecteur standard proposée par la bibliothèque python. En revanche, nous n'avons pas trouvé de SVM suffisamment précise pour concurrencer les réseaux de neurones sélectionnés.

Pour les forêts aléatoires, nous avons utilisé une méthode RandomSearch sur l'intégralité du domaine. Cette recherche n'a pas été concluante. En effet, le score de la meilleure forêt aléatoire trouvée était de 0.7546 avec pour paramètres optimaux 'nestimators' : 800, 'minsamplessplit' : 2, 'minsamplesleaf' : 1, 'maxfeatures' : 'auto', 'maxdepth' : None, 'bootstrap' : True. Ce résultat a été obtenu en explorant 400 configurations différentes. Le nombre d'arbres (nestimator) était compris entre 100 et 1000, la profondeur des arbres (maxdepth) entre 10 et 100. Les hyperparamètres de type catégorie pouvaient prendre toutes les valeurs possibles. Nous avions également des contraintes sur les noeuds et les feuilles des arbres qui allaient des contraintes standard de la bibliothèque python à des contraintes plus souples (pour minsamplesplit et minsamplesleaf). Nous n'avons pas poursuivi au delà cette recherche.

Performance finale du modèle

Le modèle choisi pour l'émulateur sera donc (cf. Annexe) un réseau de neurones à une couche cachée de 100 neurones et avec une fonction d'activation ReLU. Il sera entraîné par une méthode L-BFGS.

Modalités du test

Modèle physique utilisé	Réseau de neurone utilisé
Méthode : point fixe Critère d'arrêt : $\lvert R_{n+1} - R_n \rvert < 10^{-3}$ Nombre maximal d'itérations : 200	MLPRegressor (réseau de neurone le plus général) Une couche cachée à 50 neurones Nombre maximal d'itérations(entraînement) : 50000 Solver : L-BFGS Fonction d'activation : ReLU (par défaut)

Temps d'exécution minimal obtenu sur 2×1000 itérations de calcul du ROS sur le jeu de données simulées `Jeu_donnees_out_128.csv`

```
test_Physique: 0.0010552999999999813
test_ResNeur: 0.0004084999999918182
```

Notre émulateur est donc au moins 2 fois plus rapide que le modèle physique d'origine. Plusieurs explications sont possibles. Tout d'abord, la plupart des opérations d'algèbre linéaire sont fortement optimisées. Ensuite nous avons utilisé comme fonction d'activation la fonction *ReLU*. Celle-ci est particulièrement efficace à calculer. Le modèle physique, pour sa part, doit effectuer plusieurs itérations (que ce soit dans une méthode du point fixe ou une méthode de Newton) et doit calculer plusieurs fonctions non triviales (fonctions trigonométriques, exponentielles, puissances non entières). Une telle différence de rapidité, au prix d'une précision légèrement réduite n'est donc pas surprenante.

Pour finir, cette différence est peut-être due à l'optimisation des bibliothèques de machine learning en Python. Cette hypothèse est limitée par la simplicité du code du modèle physique mais mériterait d'être testée (passage en C++).

3.2 Approche autour de l'incertitude

Recherche d'une fonction coût approprié

Nous évoquions dans notre rapport intermédiaire la recherche d'une fonction coût dont la minimisation conduirait à l'apprentissage du α-ième quantile. Nous avions proposés plusieurs exemples qui convenaient ponctuellement (estimation du α-ième quantile pour une variable aléatoire dont on effectue des tirages répétés et indépendants) mais dont nous avions soulevé les limites dans un cadre prédictif où le α-ième quantile peut dépendre des paramètres physiques locaux.

Il se trouve qu'une des fonctions à laquelle nous avions pensé initialement convient : il s'agit de la *pinball loss*. Notons $q_\alpha(x)$ le quantile estimé, $y(x)$ une valeur cible (vitesse dans notre cas, avec x les paramètres physiques locaux). La perte s'exprime : $(y(x) - q_\alpha(x))\alpha$ si $y(x) > q_\alpha$ et $(q_\alpha(x) - y(x))(1-\alpha)$ si $y(x) \leq q_\alpha(x)$

La bibliothèque de régression quantile classique en python utilise cette fonction de perte. Et elle suppose une relation linéaire entre les paramètres physiques et le α-ième quantile.

Nous n'avons pas eu le temps d'implémenter un modèle plus général que linéaire. Il aurait été possible d'utiliser un réseau de neurone dont la fonction de perte aurait été la *pinball loss*. Cela nous aurait permis, en théorie d'exprimer plus fidèlement une prédiction pour des quantiles qui nous intéressent.

Régression quantile

Cependant, nous avons utilisé le module *QuantileRegressor* de *Scikit-learn* pour évaluer, en premier lieu si une telle méthode était envisageable. Au bout de quelques minutes, sur les plus gros jeux de données, le programme cesse et annonce ne pas disposer de mémoires suffisante. En effet, le programme génère une matrice de très grande taille (60000 par 30000 réels codés sur 64bits). Sur les jeux de données de petite taille ou de taille moyenne, l'algorithme converge rapidement et semble fournir des résultats cohérents à défaut d'être parfaits. Nous continuons pour combiner directement cet outil au réseau de neurone principal et disposer donc (modulo le choix des incertitudes relatives sur les mesures) d'un ensemble prédiction plus incertitude.

Classification Soft-Max

Nous avons essayé plusieurs géométries et plusieurs hyper-paramètres. A ce jour, nous n'arrivons pas à faire converger les algorithmes. Nous continuons à tenter d'optimiser les hyperparamètres pour arriver à une solution.

3.3 Approche satellite

Jeu de données

À partir de données satellites de la NASA, nous avons réussi à isoler des feux, et observer leur évolution dans le temps.

Pour une prise satellite entre juillet et août 2021, en Californie, environ 300 000 points sont détectés. L'algorithme de clustering a permis d'isoler environ 50 gros feux (se propageant dans une zone d'au moins 15 km de côté).

Pour chaque feu, nous disposons de 4 prises satellites par jour : deux le matin et deux le soir, espacées de 1h42, et chaque feu isolé s'étend sur au moins 10 jours.

Cela nous permet donc, à partir d'un seul fichier de points, d'obtenir plus de 1000 données d'évolution d'un feu entre des instants t et $t + \delta t$, avec $\delta t = 102$ minutes.

Nous avons également utilisé plusieurs techniques pour démultiplier la taille du jeu de données. En effet, le mécanisme de propagation du feu ne dépend que de la géographie du terrain et de sa position, pas de la "position" dans le réseau de neurone. Autrement dit, on peut translater et faire subir une rotation à chacun des feux de notre jeu de données.

Figure 23 – Evolution d'un feu de forêt, situation générale

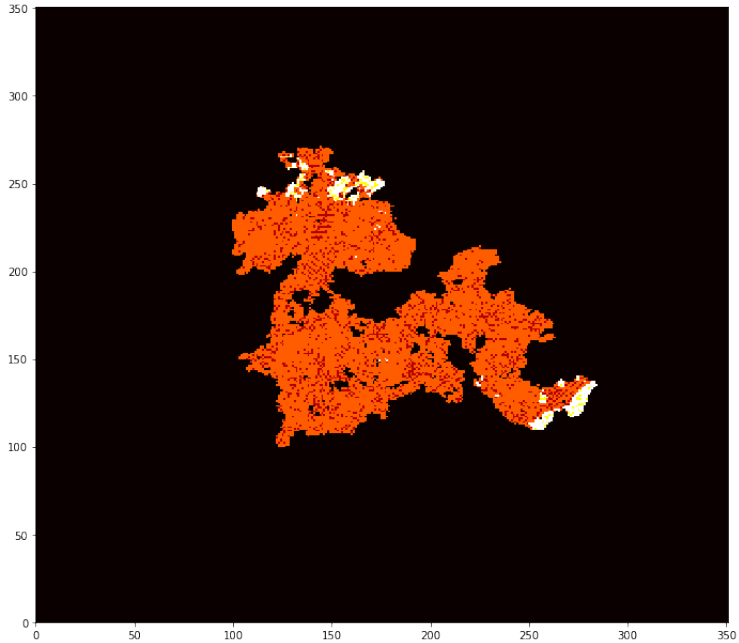
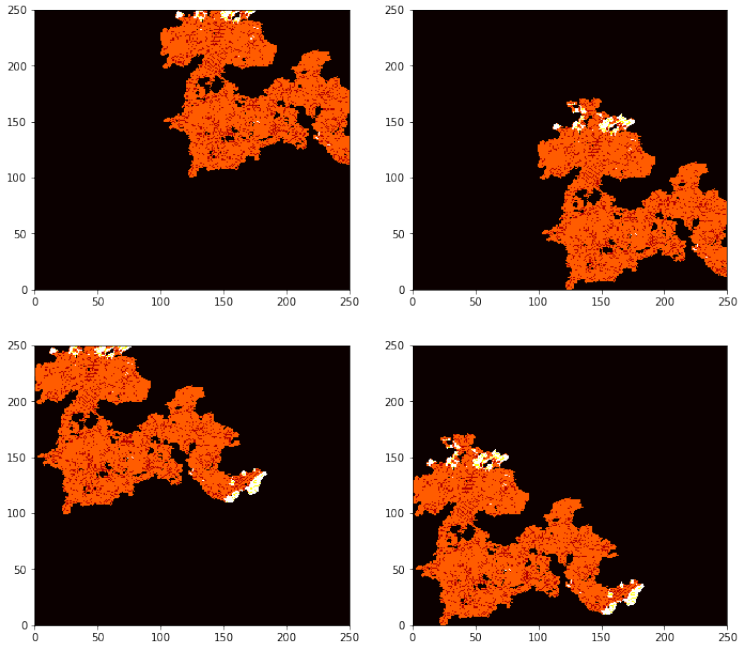


Figure 24 – Evolution d'un feu de forêt, quatre exemples de points possibles dans notre jeu de données



La seconde étape consistait en la récupération automatique de données météorologiques et géographiques dans les zones et périodes des feux obtenus. Comme expliqué en 2.6, cette étape a débuté tard et n'était pas initialement prévu, elle a abouti mais trop tard pour permettre un entraînement d'un modèle de machine learning, entraînement qui aurait de toute façon nécessité

une puissance de calcul supérieure à celle dont nous disposions.

Nous souhaiterions continuer à travailler sur ce sujet car nous pensons qu'il peut aboutir à des résultats intéressants : en se basant sur des feux réels, un modèle d'apprentissage va intégrer des phénomènes physiques (dont les phénomènes physiques extrêmes) difficiles à modéliser mathématiquement.

4 Conclusion

4.1 Conduite du projet

Au cours du projet, nous avons fait face à plusieurs difficultés présentées précédemment. Certaines ont été surmontées, notamment grâce à l'aide de notre coordinateur et de notre tuteur, mais c'est principalement le temps qui a été un facteur limitant dans l'avancée de notre projet.

L'apprentissage du C++ a notamment occupé une partie non négligeable de notre travail préliminaire, mais il ne s'est pas révélé être si utile, car nous nous sommes principalement recentré vers du code en Python.

Nous avons aussi dû acquérir des connaissances en Machine Learning pendant l'année, et les suggestions de nos encadrants nous ont poussé à nous renseigner sur un large éventail de modèles d'apprentissage.

Enfin, en parallèle des travaux principaux, nous nous sommes aussi tourné en février vers l'approche satellite, que nous n'avons malheureusement pas pu terminer mais qui nous a permis d'obtenir des données réelles. En effet, nous n'avons pas pu récupérer assez de données réelles ou des données réelles qui correspondaient à notre format d'apprentissage. Un travail supplémentaire, par un autre groupe de PSC l'année prochaine, pourrait permettre soit de poursuivre la voie "purement satellite" soit d'extraire de données satellites des informations sur la *RoS*. Cette dernière option paraît difficile selon notre tuteur.

Si le projet a légèrement évolué entre les objectifs initiaux fixés et les résultats finaux, nous avons réussi un de nos objectifs premier : faire un émulateur du modèle physique *Balbi2020* en utilisant un modèle d'apprentissage statistique, **émulateur plus rapide que le modèle physique de base. Cela permet donc de réaliser plus de simulations, facteur non négligeable dans la modélisation des risques et un des deux enjeux majeurs de la modélisation des feux de forêts.**

Nous avons également conduit **une recherche assez fine des hyperparamètres optimaux pour le problème**. Ce faisant, nous avons **confirmé plusieurs intuitions générales** sur le rôle des différents hyperparamètres et le choix optimal pour une situation donnée (profondeur, largeur, fonction d'activation...).

4.2 Travail en équipe

Ce projet fut très formateur pour nous, non seulement du point de vue académique mais aussi du point de vue de la découverte du travail en équipe sur un projet long. En effet, les intervalles de temps séparant les différentes échéances et l'ambition du projet nous ont forcé à maintenir un travail constant au cours de l'année, et une organisation sérieuse afin de ne pas faire face a des problèmes inattendus au dernier moment.

S'il était parfois difficile de nous coordonner à cinq, les recherches effectuées et la lecture d'articles scientifiques nous ont permis d'acquérir de solides connaissances en traitement de données, modélisation et Machine Learning qui nous seront utiles pour la suite de notre scolarité et confortent certains membres dans l'intérêt qu'ils portent à ces domaines.

Ce PSC nous a également initié au travail de recherche, avec la présence d'un tuteur dont l'expérience et les conseils nous ont éclairé, notamment après le rendu du rapport intermédiaire.

Pour finir, nous avons exploré, chacun notre tour au cours de l'année, différents aspects du "cycle de la donnée", depuis la collecte de données brutes, leur traitement, une première analyse avec un modèle ad hoc et enfin une optimisation avec réflexion finale.

4.3 Perspectives

Dans une optique de remplir l'objectif initial du projet qui est de fournir un outil opérationnel pour les forces d'intervention, nous allons (après avoir peaufiné notre modèle) rentrer à nouveau en contact avec l'UIISC5, à travers le membre du groupe y ayant effecté son stage de première année.

Ainsi, on pourrait imaginer une phase de test en entraînement pour identifier des pistes d'amélioration, puis potentiellement une utilisation en intervention si les critères de qualité du modèle sont validés par l'unité.

Bibliographie

[1] Martin Ester et al. “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. In : *KDD*. 1996.

[2] Sepp Hochreiter. “The Vanishing Gradient Problem during Learning Recurrent Neural Nets and Problem Solutions”. In : 6.2 (1998).

[3] G. L.W. Perry. “Current approaches to modelling the spread of wildland fire: a review”. In : *Progress in Physical Geography: Earth and Environment* 22.2 (1998), p. 222-245.

[4] Albright D. et Meisner B.N. “Classification of Fire Simulation Systems”. In : *Fire Management Notes* 59(2) (1999), p. 5-13.

[5] Pastor E. “Mathematical models and calculation systems for the study of wildland fire behaviour”. In : *Progress in Energy and Combustion Science* (2003), p. 138-153.

[6] Matt Pharr et Greg Humphreys. “Physically Based Rendering : From Theory To Implementation”. In : 7.4. Halton sampler (2004).

[7] Deian Stefan Ishaan L. Dalal et Jared Harwayne-Gidansky. In : *The Cooper Union for the Advancement of Science and Art* (2008).

[8] Xingjian Shi et al. *Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting*. 2015.

[9] Jan Kukačka, Vladimir Golkov et Daniel Cremers. *Regularization for Deep Learning: A Taxonomy*. 2017.

[10] Philipp Probst, Marvin N. Wright et Anne-Laure Boulesteix. “Hyperparameters and tuning strategies for random forest”. In : 9.3 (jan. 2019).

[11] Jacques Henri Balbi et al. “A convective–radiative propagation model for wildland fires”. In : (2020).

[12] Tong Yu et Hong Zhu. *Hyper-Parameter Optimization: A Review of Algorithms and Applications*. 2020.

[13] Université de Corse. *Portail GitHub, code du modèle Balbi2020*. `https://github.com/forefireAPI/firefront`. Mis à jour en 2021.

[14] Meteostat Developers. *Documentation of the python Library Meteostat*. `https://dev.meteostat.net/python`. Mis à jour en 2022.

[15] NASA. *Fire Information for Resource Management System*. `https://firms.modaps.eosdis.nasa.gov/download/`. Données mises à jour tous les jours.

Annexes

1.Choix des hyperparamètres

Table 2: Impact de la géométrie du réseau sur le score pour le jeu de données à 2^{15} occurrences. Les autres hyper-paramètres restant constants

Géométrie	Fonction d'activation	score
(20)	ReLU	0.954
(100)	ReLU	0.971
(100)	TanH	0.981
(200)	ReLU	0.975
(20,20)	ReLU	0.946
(50,50)	ReLU	0.975
(100,100)	ReLU	0.937
(20,20,20)	ReLU	0.962

On remarque ici qu'un réseau de neurone plus profond n'est pas forcément meilleurs voir pire à largeur égale. A noter que le troisième choix d'hyperparamètres est le meilleur obtenu lors de la phase d'optimisation de ceux-ci.

Table 3: Impact de la géométrie du réseau sur le score pour le jeu de données à 2^8 occurrences. Les autres hyper-paramètres restant constants

Géométrie	Activation	score
(20)	ReLu	-0.043
(100)	ReLU	0.152
(200)	ReLU	0.204
(20)	Tanh	-2.173
(100)	Tanh	-1.494
(200)	Tanh	-0.974
(20, 20, 20)	ReLU	0.836
(20, 20, 20)	Tanh	-0.312
(50, 50, 50)	ReLU	0.719
(50, 50, 50)	Tanh	0.194
(100, 100, 100)	ReLU	-0.139
(100, 100, 100)	Tanh	-0.425
(50, 50, 50, 50)	ReLU	0.132
(50, 50, 50, 50)	Tanh	0.392

Table 3: Impact de la géométrie du réseau sur le score pour le jeu de données à 2^8 occurrences. Les autres hyper-paramètres restant constants

Géométrie	Activation	score
(20, 20, 20, 20)	ReLU	0.734
(20, 20, 20, 20)	Tanh	0.593
(30, 30, 30)	ReLU	0.0410
(20, 20, 20)	ReLU	0.837
(40, 40, 40)	ReLU	0.346
(35, 35, 35)	ReLU	0.406
(45, 45, 45)	ReLU	−0.093

Nous avons en plus retiré du tableau précédent les géométries à une couche cachée qui, avec d'autres *solver* donnaient un score négatif. Les scores très petits voire négatifs prouvent que la géométrie à une couche cachée est inadaptée. On remarque immédiatement que l'algorithme réussit à converger pour des valeurs bien particulières alors que pour d'autre le résultat est inopérant. On remarque enfin que ReLU donne en moyenne de meilleurs résultats que la fonction tangente hyperbolique.

Table 4: Impact du solver (et de la largeur de la couche cachée) sur le score. Pour les tout petits jeux de données, le cas a déjà été traité

Taille du jeu de données	Géométrie	Activation	Solver	score
32768	(20)	ReLU	SGD	0.797
32768	(20)	ReLU	Adam	0.811
32768	(20)	ReLU	L-BFGS	0.954
32768	(100)	ReLU	SGD	0.792
32768	(100)	ReLU	Adam	0.816
32768	(100)	ReLU	L-BFGS	0.957
32768	(200)	ReLU	SGD	0.813
32768	(200)	ReLU	Adam	0.824
32768	(200)	ReLU	L-BFGS	0.975
4096	(20)	ReLU	SGD	0.618
4096	(20)	ReLU	Adam	0.641
4096	(20)	ReLU	L-BFGS	0.806
4096	(100)	ReLU	SGD	0.708
4096	(100)	ReLU	Adam	0.648
4096	(100)	ReLU	L-BFGS	0.653
4096	(200)	ReLU	SGD	0.625
4096	(200)	ReLU	Adam	0.641

Table 4: Impact du solver (et de la largeur de la couche cachée) sur le score. Pour les tout petits jeux de données, le cas a déjà été traité

Taille du jeu de données	Géométrie	Activation	Solver	score
4096	(200)	ReLU	L-BFGS	0.628
16384	(20)	ReLU	SGD	0.790
16384	(20)	ReLU	L-BFGS	0.908
16384	(100)	ReLU	SGD	0.722
16384	(100)	ReLU	Adam	0.824
16384	(100)	ReLU	L-BFGS	0.957
16384	(200)	ReLU	SGD	0.840
16384	(200)	ReLU	Adam	0.854
16384	(200)	ReLU	L-BFGS	0.975

On observe que, à nombre d'itération égal, les solvers *Stochastic Gradient Descent* et *Adam* sont moins performants en général. Cette différence se perçoit surtout quand L-BGFS a réussit à trouver un minimum "acceptable" pour une régression déterministe (score plus élevé que 0.95 par exemple). Cette différence n'est pas due à une différence d'interprétation ou d'utilisation du maximum d'itération dans la phase d'apprentissage puisque même en augmentant ce nombre d'itération, *Adam* et *SGD* réalisent des scores identiques.

Table 5: Impact du paramètre de régularisation pour le jeu de données à 2^8 ocurrences

Géométrie	Paramètre de régularisation	score
(20)	0.0001	-0.0720
(20)	0.001	0.0128
(20)	0.01	-0.001
(20)	0.1	-0.336
(100)	0.0001	0.167
(100)	0.001	0.148
(100)	0.01	-0.304
(100)	0.1	-0.372
(200)	0.0001	0.228
(200)	0.001	0.234
(200)	0.01	-0.262
(200)	0.1	-0.294
(50, 50)	0.0001	0.376
(50, 50)	0.001	0.617
(50, 50)	0.01	-0.586
(50, 50)	0.1	-0.634

Table 5: Impact du paramètre de régularisation pour le jeu de données à 2^8 ocurrences

Géométrie	Paramètre de régularisation	score
(50, 50, 50)	0.0001	0.744
(50, 50, 50)	0.001	0.662
(50, 50, 50)	0.01	-0.649
(50, 50, 50)	0.1	-0.574
(20, 20, 20)	0.0001	0.669
(20, 20, 20)	0.001	0.433
(20, 20, 20)	0.01	-0.346
(20, 20, 20)	0.1	-0.916
(100, 100, 100)	0.0001	-0.0330
(100, 100, 100)	0.001	-0.184
(100, 100, 100)	0.01	-0.735
(100, 100, 100)	0.1	-0.607
(20, 20, 20, 20)	0.0001	-0.267
(20, 20, 20, 20)	0.001	0.747
(20, 20, 20, 20)	0.01	0.422
(20, 20, 20, 20)	0.1	-0.821

Nous observons que pour certaines géométries (4 couches cachées à 20 neurones par exemple) et pour certaines valeurs spécifiques du paramètre de régularisation, une amélioration marginale peut être remarquée. Sur d'autres cette amélioration est plus visible. Cependant, dans l'immense majorité des cas, soit on passe d'un régresseur très mauvais (score nul voire négatif) à un régresseur mauvais (score inférieur à 0.7 alors même que l'on travaille sur un jeu de donnée déterministe) soit l'augmentation du paramètre a un effet négatif sur le score.

2.Modèle physique, notations

Table 6: Constantes

Notation	Nom	Valeur
B	Stefan–Boltzmann constant ($W.m^{-2}.K^{-4}$)	$5.6\dot{1}0^{-8}$
C_{pw}	Specific heat of water ($J.kg^{-1}.K-1$)	4180
C_{pa}	Specific heat of air ($J.kg^{-1}.K^{-1}$)	1150
g	Acceleration due to gravity ($m.s^{-2}$)	9.81
K_1	Drag coefficient ($s.m^{-1}$)	130
p	Proportionality coefficient $\left(= \frac{2}{r_{00}\tau_0}\right)$	
r_{00}	Model coefficient	$2.5\dot{1}0^{-5}$
s_t	Air–pyrolysis gas mass ratio in the flame body	17

Table 6: Constantes

Notation	Nom	Valeur
T_a	Ambient temperature (K)	300
T_i	Ignition temperature (K)	600
T_{vap}	Vaporisation temperature (K)	373
χ_0	Radiative factor	0.3
Δ_H	Heat of combustion of the pyrolysis gases $(J.kg^{-1})$	$1.74\dot{1}0^7$
Δ_h	Heat of latent evaporation $(J.kg^{-1})$	$2.3\dot{1}0^6$
ρ_a	Air density at ambient temperature $(kg.m^{-3})$	1.184
τ_0	Flame residence time parameter $(s.m^{-1})$	75 591

Table 7: Paramètres physiques locaux décrivant le terrain

Notation	Description
A	Radiation coefficient
C_p	Specific heat of fuel $(J.kg^{-1}.K^{-1})$
h	Fuel bed depth (m)
m	Fuel moisture content
q	Ignition energy $(J.kg^{-1})$
s	Surface area-to-volume ratio of fine fuel (m^{-1})
S	Leaf area per square metre $(m^2.m^{-2})$
U_0	Velocity of the fire-generated inflow coming from the unburnt area $(m.s^{-1})$
α	Terrain slope angle (deg)
β	Packing ratio
ρ_v	Fuel particle density $(kg.m^{-3})$
σ	Dead fuel load $(kg.m^{-2})$

Table 8: Termes de calcul

Notation	Description
H	Flame height (m)
R	Rate of fire spread $(m.s^{-1})$
T	Mean flame temperature (K)
U	Sum of normal $(m.s^{-1})$
u_0	Upward gas velocity at flame body mid-height on flat terrain $(m.s^{-1})$
γ	Flame tilt angle (deg)
χ	Radiative fraction